Cost-Efficient Retrieval-Augmented Generation (RAG) System Using Local LLMs

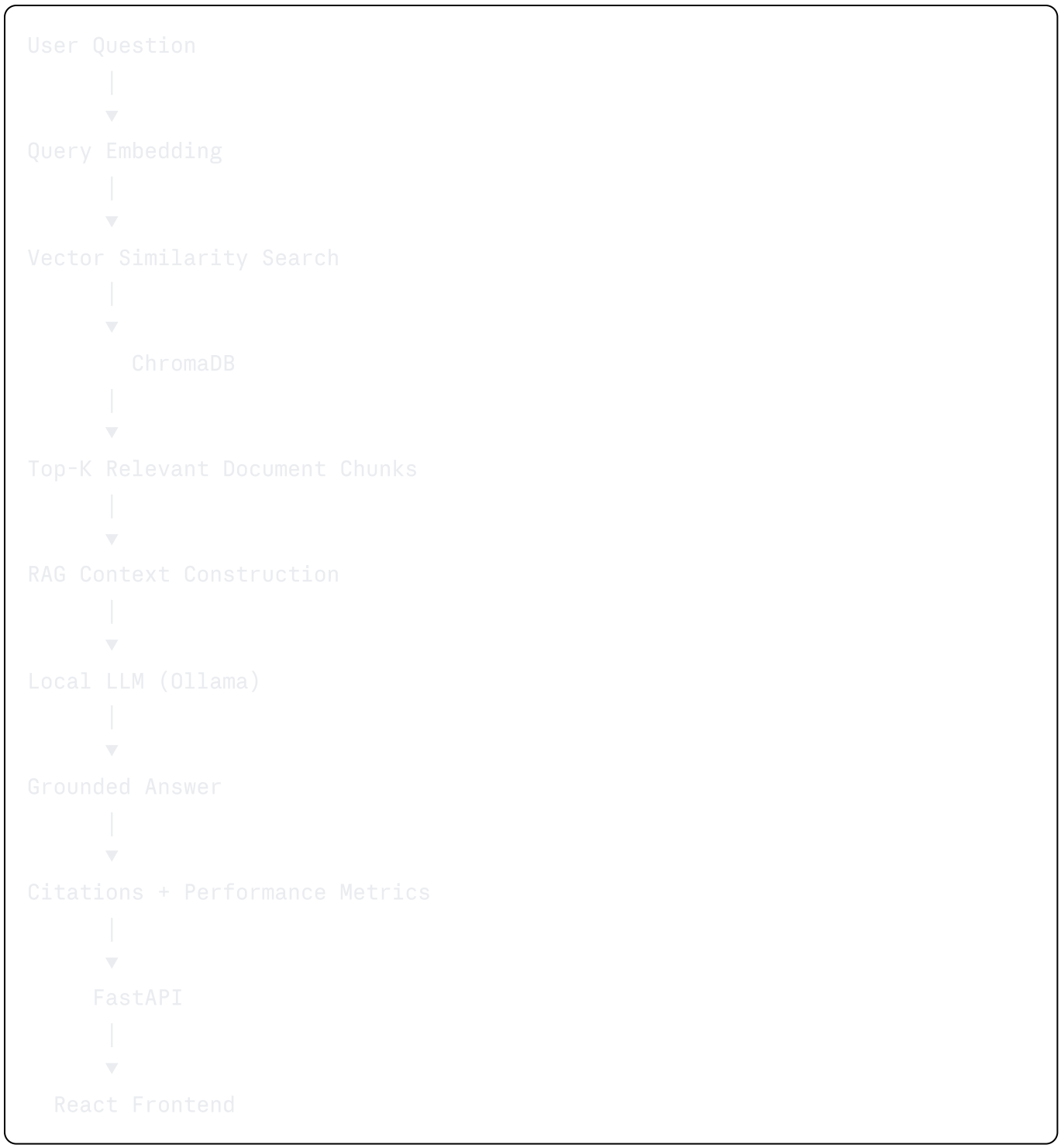
Project Documentation Report

CHAPTER 1 — INTRODUCTION

1.1 Project Overview

This project implements a **Retrieval-Augmented Generation (RAG)** system designed to answer natural-language questions using a locally indexed collection of Python documentation. Retrieval-Augmented Generation is a technique in which a language model's answer is grounded in relevant text retrieved from an external knowledge base, rather than relying solely on the model's internal (parametric) knowledge.

The system follows the pipeline below:



The knowledge base indexed by the system consists of **Python documentation stored in HTML files.** The system can answer questions such as "What is a Python class?", "How does inheritance work in Python?", and "What is a lambda expression?", grounding each answer in retrieved source content and returning citations.

The system was also tested with an out-of-domain question — "What is the capital of France?" — to confirm that it correctly recognizes when the indexed knowledge base does not contain relevant information, rather than fabricating an answer.

1.2 Background

Large Language Models (LLMs) are trained on a fixed corpus of data up to a certain point in time and do not automatically have access to a specific, private, or narrow documentation set. When such a model is asked a question about a specific technical domain, it may answer from general knowledge, which can be imprecise, outdated, or simply incorrect (a

phenomenon known as **hallucination**). Retrieval-Augmented Generation addresses this by supplying the model with the most relevant passages from a trusted document collection at query time, so that the generated answer is grounded in verifiable source material.

1.3 Problem Statement

Ordinary language models face several limitations when used to answer questions about a specific documentation set:

- They may not have access to the exact or most current documentation relevant to the query.
- They generate answers based on general training knowledge rather than a specific source.
- They may hallucinate plausible-sounding but incorrect information.
- Manually searching through large documentation collections is slow and inefficient for the end user.
- Routing every question to a paid, cloud-hosted LLM API introduces a recurring per-request cost.
- A domain-specific question-answering system needs reliable, traceable access to the underlying source material.

RAG addresses these problems by first retrieving the most relevant chunks of documentation using semantic (embedding-based) search, and then supplying those chunks as context to the language model so that the generated answer is grounded in retrieved evidence rather than pure recall from model parameters.

1.4 Motivation

The project was undertaken with the following motivations:

- To build a practical, working RAG system rather than a simple chatbot wrapper.
- To gain a thorough, hands-on understanding of the complete RAG pipeline — from raw documents to a served answer.
- To reduce dependence on paid, external LLM APIs.
- To run language-model inference locally using Ollama.
- To ground generated responses in source documentation rather than the model's unconstrained internal knowledge.
- To provide source citations so that answers are traceable and verifiable.
- To evaluate retrieval quality quantitatively using standard information-retrieval metrics.
- To expose the system as a proper HTTP API using FastAPI.
- To build a real, professional frontend application rather than relying on a quick prototyping tool such as Streamlit.
- To understand the interaction between embeddings, vector databases, retrieval, and generation as parts of one integrated system.

1.5 Objectives

The primary objective of the project is to build a complete Retrieval-Augmented Generation pipeline that:

1. Loads documentation from source files.
2. Processes and cleans the extracted text.
3. Splits the documents into manageable chunks.
4. Generates vector embeddings for each chunk.
5. Stores the embeddings in ChromaDB.
6. Converts user questions into query embeddings.
7. Retrieves the most relevant chunks for a given question.
8. Provides the retrieved chunks as context to a local LLM.
9. Generates a grounded answer using that context.
10. Provides citations corresponding to the retrieved chunks used.
11. Measures retrieval and generation latency.
12. Exposes the complete RAG system through a FastAPI backend.
13. Provides a professional React-based frontend for interacting with the system.
14. Avoids the recurring costs associated with paid external LLM APIs by performing inference locally.

1.6 Scope

The current project scope covers:

- Python documentation as the indexed knowledge domain.
- HTML document ingestion.
- Document chunking.
- Vector embedding generation.
- ChromaDB vector storage.
- Semantic (embedding-based) retrieval.
- Local LLM inference through Ollama.
- RAG-based answer generation.
- Citation generation.
- Quantitative retrieval evaluation.
- A FastAPI backend.
- A React/Vite frontend.

The project is, at its current stage, a **documentation-based RAG assistant** focused on a specific, indexed knowledge base. It is **not** a general-purpose internet search engine, and it does not perform live web search or automatic ingestion of new documents.

1.7 Key Features

- End-to-end RAG pipeline from raw HTML documentation to a generated, cited answer.
- Fully local inference — no paid external LLM API is required at query time.
- Quantitative retrieval evaluation using a hand-built ground-truth dataset.
- Citation support that links generated answers back to specific retrieved chunks.
- Adjustable Top-K retrieval, configurable from the frontend.
- A modular backend broken into independently testable components.
- A professional React/Vite frontend with chat-style interaction, source inspection, and live backend status monitoring.

CHAPTER 2 — TECHNOLOGY AND SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Because the language model runs locally through Ollama, the primary hardware consideration is the machine's capacity to run local LLM inference (CPU/GPU, RAM). The project does not report fixed hardware specifications as a formal requirement; instead it notes that local generation latency is directly dependent on the hardware used (see Chapter 11 and Section on Performance Discussion).

2.2 Software Requirements

Component	Requirement
Backend language	Python
Backend framework	FastAPI
ASGI server	Uvicorn
Vector database	ChromaDB
Local LLM runtime	Ollama
Frontend framework	React (with Vite)
Package manager (frontend)	npm
Development environment	VS Code
Python environment management	Conda environment

2.3 Technology Stack

Layer	Technology	Purpose
Backend	Python	Core implementation language
Backend	FastAPI	HTTP API framework exposing the RAG pipeline
Backend	Uvicorn	ASGI server running the FastAPI application
RAG Orchestration	Custom `RAGPipeline`	Coordinates retrieval and generation
RAG Orchestration	`Retriever`	Performs semantic retrieval of relevant chunks
RAG Orchestration	`EmbeddingModel`	Converts text into vector embeddings
Vector Storage	ChromaDB	Stores and searches document embeddings
Generation	Ollama	Runs the local LLM used for answer generation
Document Processing	HTML documentation	Source knowledge base
Document Processing	Custom loaders	Extract text from HTML source files
Document Processing	Chunking utilities	Split documents into retrievable chunks
Frontend	React	UI framework
Frontend	Vite	Frontend build/dev tooling
Frontend	JavaScript	Application logic
Frontend	CSS	Styling
Frontend	lucide-react	Icon library
Data	Indexed Python documentation	Knowledge base content
Data	ChromaDB vector collection	Persisted embedding store
Evaluation	Ground-truth JSON	Reference answers used for retrieval evaluation

Layer	Technology	Purpose
Evaluation	Recall@1, Recall@3, Recall@5, MRR, nDCG@5	Retrieval quality metrics
Development Environment	VS Code	Code editor
Development Environment	Conda environment	Python dependency/environment isolation

2.4 Libraries and Frameworks

The backend is built around a small set of custom modules (`loaders.py`, `chunker.py`, `embeddings.py`, `vector_store.py`, `retriever.py`, `generator.py`, `rag.py`, `api.py`) rather than a large third-party orchestration framework, giving the project a modular, independently testable structure (see Chapter 3 and Chapter 9 for details). FastAPI and Uvicorn form the web-serving layer; ChromaDB is the vector storage layer; Ollama provides local LLM inference; and React with Vite forms the client application layer.

CHAPTER 3 — SYSTEM ANALYSIS AND DESIGN

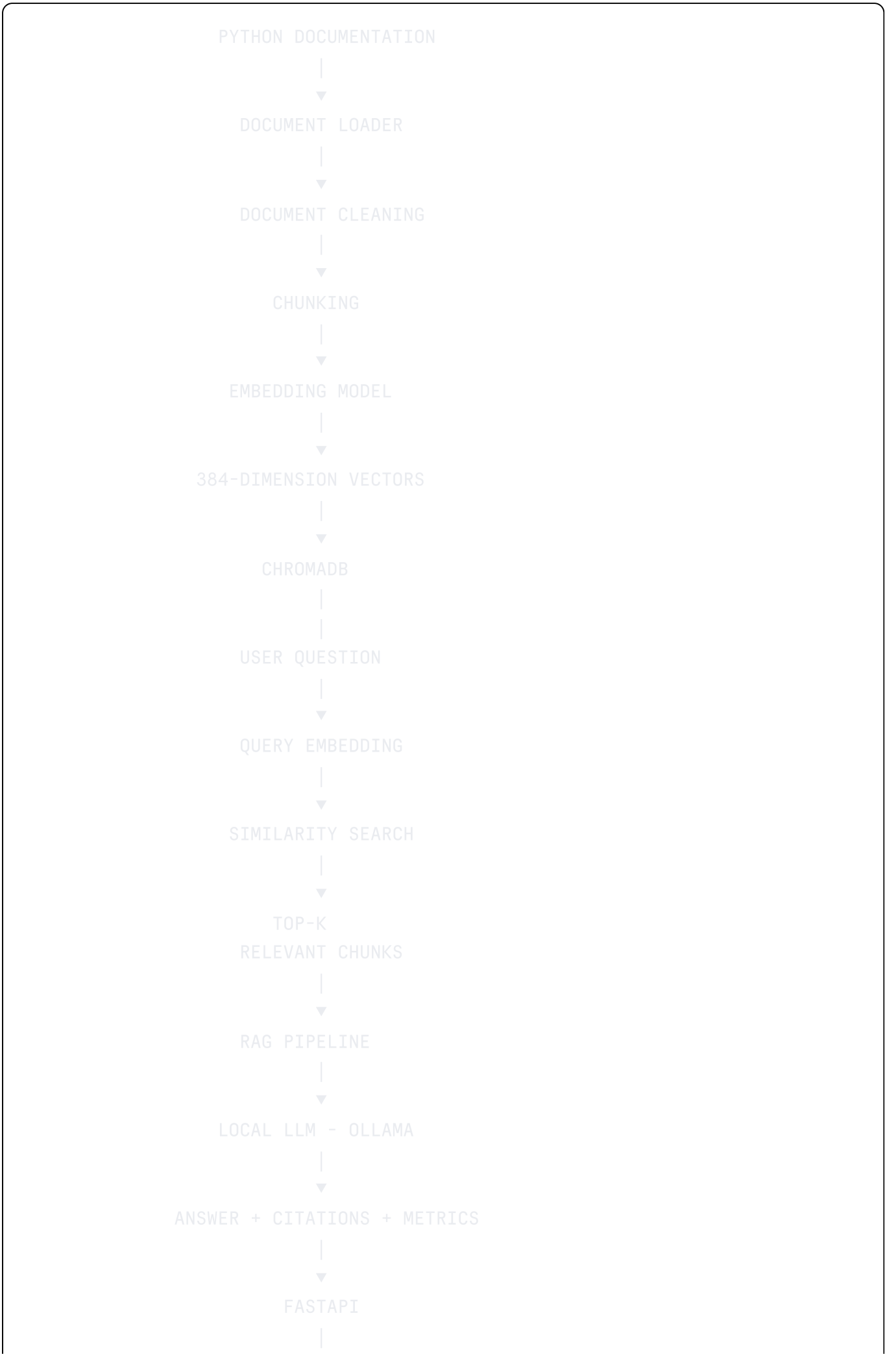
3.1 Existing Approach

A common existing approach to building a documentation question-answering assistant is to send the user's question, together with a large portion of documentation or the entire model context, directly to a general-purpose LLM — frequently a paid, cloud-hosted API such as OpenAI. This approach has two key drawbacks: it incurs a recurring cost for every request, and it does not guarantee that the model's answer is actually grounded in the specific documentation set, since the model may fall back on its own general training knowledge.

3.2 Proposed System

The proposed system instead separates the problem into two stages: **retrieval** and **generation**. Relevant chunks of the indexed documentation are first retrieved using semantic vector search, and only those chunks — rather than the entire documentation set — are passed to the language model as context. Generation is performed locally via Ollama, removing the need for a paid external LLM API while keeping answers grounded in retrieved evidence.

3.3 System Architecture



The architecture is naturally divided into an **offline/indexing stage** (top half of the diagram, executed once per document set) and an **online/query stage** (bottom half, executed per user question). This distinction is discussed further in Chapter 3.6 and Chapter 11.

3.4 System Workflow

The end-to-end user workflow is:

1. Start the FastAPI backend.
2. Start the React development server.
3. Open the frontend in the browser.
4. The frontend checks backend health via `/health`.
5. The backend status indicator becomes "Connected".
6. The user selects a suggested question or enters a custom question.
7. The user optionally adjusts the Top-K value.
8. React sends the question to FastAPI.
9. FastAPI passes the request to `RAGPipeline`.
10. A query embedding is generated for the question.
11. ChromaDB retrieves the relevant chunks.
12. The retrieved chunks are passed to Ollama as context.
13. Ollama generates the answer.
14. The backend returns the answer, citations, and performance metrics.
15. React displays the result.
16. The user can inspect the sources and copy the answer.

3.5 Module Architecture

Module	Responsibility
`loaders.py`	Document loading from HTML source files
`chunker.py`	Splitting documents into retrievable chunks
`embeddings.py`	Generating vector embeddings
`vector_store.py`	Interaction with ChromaDB
`retriever.py`	Retrieving relevant chunks for a query
`generator.py`	Generating the LLM response
`rag.py`	Orchestrating the complete RAG pipeline
`api.py`	Exposing the pipeline through FastAPI

This modular decomposition allows each stage of the pipeline to be developed, tested, and debugged independently (see Chapter 11.1).

3.6 Data Flow

Offline / Indexing stage:

```
HTML documents → Loader → Cleaning → Chunking → Embedding → ChromaDB
```

Online / Query stage:

```
User question → Query embedding → ChromaDB similarity search →
Top-K chunks → RAG prompt → Ollama → Answer → Citations → FastAPI → React
```

The indexing stage is performed once (or whenever the document set changes) and is comparatively slow, since it involves processing and embedding the entire document collection. The query stage is performed for every user question and is designed to be fast for the retrieval portion, with generation latency dominated by the local LLM (see Chapter 11 and the Performance Discussion).

CHAPTER 4 — DATA INGESTION AND PREPROCESSING

4.1 Source Documents

The knowledge base indexed by the system consists of **Python documentation stored in HTML files**. These files serve as the raw source material for the entire retrieval pipeline.

4.2 Document Loading

Custom loader code (`loaders.py`) reads the HTML documentation files from disk and prepares them for downstream processing.

4.3 Text Extraction

Relevant textual content is extracted from each HTML file, discarding markup that is not needed for retrieval.

4.4 Cleaning

The extracted text is cleaned/processed before chunking, so that chunks passed downstream are free of irrelevant formatting artifacts and are suitable both for embedding and for direct display to the user as part of a citation.

4.5 Chunking

Splitting a document into smaller chunks, rather than indexing whole documents as single units, is necessary because:

- Retrieval becomes less precise when large blocks of text are treated as a single unit.
- The context supplied to the LLM becomes unnecessarily large.
- Relevant information can be buried inside a large document and diluted by irrelevant surrounding text.
- Using the LLM's context window efficiently requires focused, relevant chunks rather than entire documents.

The chunking process preserves useful textual boundaries so that each retrieved chunk remains understandable on its own, and each chunk retains metadata that allows it to be traced back to its source document (e.g., `classes.html`, chunk index).

4.6 Metadata

Each chunk is stored with metadata identifying its source document and chunk index. This metadata is later used to construct citations (see Chapter 8.7) and to trace an answer back to its originating source material.

4.7 Document Statistics

The following are actual measured results from the implemented ingestion pipeline:

Metric	Value
Documents processed	7
Chunks processed	363
Embedding dimension	384
Vector store count	363

CHAPTER 5 — EMBEDDING AND VECTOR STORAGE

5.1 Embeddings

An **embedding** is a numerical vector representation of a piece of text, positioned in a high-dimensional space such that texts with similar meaning are located close to one another. Embeddings enable **semantic search** — finding text that is conceptually related to a query, rather than requiring an exact keyword match. This is particularly valuable for a documentation-QA system, since a user's question rarely uses the exact same wording as the source documentation, but may still be asking about the same concept.

5.2 Embedding Generation

For document chunks:

```
Text → Embedding Vector
```

For user questions at query time:

```
Question → Query Embedding
```

The system then compares the query embedding against the stored document embeddings to identify the most semantically similar chunks. The specific embedding model implementation is an internal detail of the `EmbeddingModel` component; the measured output dimensionality is reported below.

5.3 Vector Dimensions

The embedding model used in this project produces **384-dimensional vectors**.

5.4 ChromaDB

ChromaDB is used as the vector database for the project. It stores:

- Document chunks (text)
- Their embeddings (vectors)
- Associated metadata (e.g., source document, chunk index)
- Unique identifiers for each chunk

5.5 Vector Storage

The final vector store, after ingestion of all 7 source documents, contains **363 vectors**, matching the number of chunks processed.

5.6 Similarity Search

Retrieval proceeds as follows:

A lower distance score generally indicates stronger similarity for the vector-search metric configured in the system. The user can control how many chunks (Top-K) are retrieved via the frontend, which currently supports values from **1 to 10**, with a **default of 5**.

CHAPTER 6 — RETRIEVAL SYSTEM

6.1 Retriever

The `Retriever` component is the bridge between a user's natural-language question and the indexed documentation. It is responsible for:

1. Accepting a natural-language question.
2. Generating an embedding for the question.
3. Querying ChromaDB.
4. Retrieving the most relevant chunks.
5. Returning the retrieved chunks along with their metadata.

6.2 Query Embedding

Each incoming question is converted into a query embedding using the same embedding process applied to document chunks during ingestion, ensuring both are represented in the same vector space and can be meaningfully compared.

6.3 Similarity Search

ChromaDB compares the query embedding against all stored chunk embeddings and computes a similarity/distance score for each, which is used to rank the candidate chunks.

6.4 Top-K Retrieval

The system returns the **Top-K** highest-ranked chunks for a given query. Top-K is configurable from the frontend (range 1–10, default 5), allowing the user to trade off between broader context and retrieval precision.

6.5 Retrieval Results

Each retrieval result includes the chunk text, its source document, its chunk index, and its distance/similarity score, all of which are later surfaced to the user through the citation system (Chapter 8.7).

CHAPTER 7 — RETRIEVAL EVALUATION

7.1 Ground Truth

To objectively evaluate retrieval quality, a ground-truth evaluation dataset was created. The workflow used to build it was:

The resulting ground-truth file was stored at `evaluation/ground_truth.json`.

7.2 Evaluation Dataset

The ground-truth dataset consists of **20 questions**, each manually associated with the chunk(s) considered relevant to it.

7.3–7.7 Retrieval Metrics

Metric	Score	Interpretation
Recall@1	0.5750	Fraction of questions for which the single relevant chunk appears as the very first retrieved result.
Recall@3	0.8667	Fraction of questions for which the relevant chunk appears within the first three retrieved results.
Recall@5	0.9500	Fraction of questions for which the relevant chunk appears within the first five retrieved results.
MRR (Mean Reciprocal Rank)	0.7375	Measures how high, on average, the first relevant result appears in the ranked list — higher is better.
nDCG@5 (normalized Discounted Cumulative Gain)	0.7896	Measures the overall ranking quality among the top five results, giving greater weight to relevant results that are ranked higher.

7.8 Results and Interpretation

The **Recall@5 = 95%** result indicates that, for the evaluated question set, the relevant chunk was retrieved within the top five results for the large majority of questions. Recall improves sharply as K increases (57.5% at K=1 to 95% at K=5), suggesting that while the single top-ranked result is not always the correct chunk, the correct chunk is very likely present in a modestly sized retrieved set. The MRR of 0.7375 and nDCG@5 of 0.7896 further indicate that, on average, relevant chunks tend to be ranked reasonably close to the top rather than deep within the retrieved list. These results should be interpreted as evidence of **strong but not perfect** retrieval performance on the evaluated question set; they do not constitute a guarantee of retrieval accuracy for arbitrary future questions.

CHAPTER 8 — RAG GENERATION

8.1 RAG Concept

The system is called "Retrieval-Augmented Generation" because it combines two previously separate techniques: **retrieval** of relevant text from an external source, and **generation** of a natural-language answer by a language model. The generation step is conditioned ("augmented") on the retrieved text, rather than depending solely on the model's internal knowledge.

8.2 Context Construction

The chunks returned by the Retriever are assembled into a context block that is provided to the local LLM alongside the user's original question.

8.3 Prompt Construction

8.4 Ollama

Ollama is the local runtime used to serve the language model. It is a key part of the project's cost-efficiency: the project originally considered using the OpenAI API, but encountered insufficient API credits during development. Since the project's requirements allow (and in this case required) the use of a free model, the final implementation was changed to use Ollama for local LLM inference.

8.5 Local LLM

The benefits of using a locally-run LLM through Ollama, as realized in this project, are:

- No paid OpenAI API is required.
- No recurring per-request API charges.
- Inference happens entirely on the local machine.
- Greater control over where data is processed.
- Well suited to an educational/local RAG project.
- Works directly with the existing retrieval pipeline without modification to the retrieval logic.

The final generation architecture is:

```
RAG Retrieval → Retrieved Context → Ollama Local LLM → Generated Answer
```

Ollama itself is used strictly as the local LLM inference runtime; it is **not** used as an embedding database — that role is filled by ChromaDB (Chapter 5.4).

8.6 Answer Generation

Given the question and the retrieved context, the local LLM generates a grounded answer. Because generation is conditioned on retrieved chunks, the model is guided toward producing an answer consistent with the indexed documentation rather than an answer based purely on unconstrained internal knowledge.

8.7 Citations

The retrieved chunks used to generate an answer carry metadata such as:

- Source document
- Chunk index
- Distance score
- Citation number

For example:

```
classes.html
chunk 0
distance = 0.4321
```

The generated answer can reference specific sources using numbered citation markers, e.g. `[1]`, `[2]`, `[5]`. The frontend allows the user to expand a "Sources" section to inspect the retrieved chunks that contributed to a given answer, improving transparency by making the evidence behind an answer directly inspectable. Citation formatting is an area noted as suitable for future improvement (see Chapter 12.3).

8.8 Performance Metrics

For each query, the system records:

- Retrieved chunks
- Retrieval latency
- Generation latency
- Total latency
- Input tokens
- Output tokens
- Total tokens

Two illustrative examples observed during testing:

Query	Retrieved Chunks	Retrieval Latency	Generation Latency
Example A	5	~24 ms	~5596 ms
Example B	5	~23 ms	~6244 ms

These figures are illustrative examples from actual testing, not fixed benchmarks — local generation speed depends on the hardware and model used. They do, however, clearly illustrate that retrieval is comparatively fast, while local LLM generation accounts for the large majority of total response time.

CHAPTER 9 — FASTAPI BACKEND

9.1 Backend Architecture

The backend is implemented using **FastAPI**, with the main application defined in `app/api.py`. FastAPI exposes the underlying `RAGPipeline` (Chapter 8) as a set of HTTP endpoints that the React frontend consumes.

9.2 API Structure

Method	Endpoint	Purpose
GET	`/`	Confirms that the API application is running
GET	`/health`	Reports backend availability to the frontend
POST	`/query`	Submits a question to the RAG pipeline and returns the generated answer

9.3 `/` Endpoint

A simple GET endpoint used to confirm that the FastAPI application is running.

9.4 `/health` Endpoint

Used by the frontend to determine whether the backend is available. Example response:

```json
{
    "status": "healthy"
}
```

9.5 `/query` Endpoint

Accepts a JSON request of the form:

```json
{
    "question": "What is a Python class?",
    "top_k": 5
}
```

The endpoint calls the RAG pipeline:

```python
rag.query(
    question=request.question,
    top_k=request.top_k
)
```

and returns the generated result, including the answer, citations, retrieved chunks, and performance metrics.

9.6 Request Model

The request body is validated using a Pydantic model:

```python
class QueryRequest(BaseModel):
    question: str
    top_k: int = 5
```

9.7 Response Structure

The backend response includes:

- `question`
- `answer`
- `citations`
- `retrieved chunks`
- `retrieval latency`
- `generation latency`
- `token counts`

This structured response allows the frontend to present both the generated answer and the underlying system performance in a single view.

9.8 API Testing

The FastAPI Swagger/OpenAPI interface was used to test the API directly, prior to frontend integration.

Endpoint	Result
`GET /health`	HTTP 200 — `{"status": "healthy"}`
`POST /query`	HTTP 200 — returned generated answer, citations, and metrics

This confirmed successful backend operation before frontend integration began.

CHAPTER 10 — REACT FRONTEND

10.1 Frontend Architecture

The frontend does **not** use Streamlit in the final implementation. It was deliberately built using React, Vite, JavaScript, CSS, and the `lucide-react` icon library, in order to produce a more professional and engaging interface than a basic Streamlit prototype would allow.

10.2 React + Vite

Vite provides a lightweight, modern development environment for the React application, used both for local development (`npm run dev`) and as the frontend build tool.

10.3 Sidebar

The sidebar contains:

- Project branding
- A "New Chat" button
- A list of recent chats
- The configured backend URL
- The Top-K control
- Backend connection status
- A ChromaDB indicator
- An Ollama indicator
- A note that no paid API is required

10.4 Chat System

The React application supports:

- Starting new conversations
- Recent chat titles
- An active chat concept
- User messages
- Assistant (RAG-generated) messages
- Error messages
- Multiple sequential questions within a conversation
- Copying an answer
- Expanding/collapsing the sources panel

Chat state is currently maintained using **React state** on the client. The current chat history is frontend state only — it is not backed by a persistent database, and this is explicitly noted as a current limitation (see Chapter 12.2) rather than a missing production feature.

10.5 Suggested Questions

The frontend provides a set of predefined suggested questions, allowing a user to quickly test the application without composing a question manually:

- "What is a Python class?"
- "How does inheritance work in Python?"
- "What is a lambda expression?"
- "How does Python handle exceptions?"
- "How can a list be used as a stack?"
- "What is the difference between a list and a tuple?"
- "How are dictionaries used in Python?"
- "How does Python search for modules?"
- "How can data be written to a file in Python?"
- "What are generator expressions in Python?"

10.6 Top-K Control

The sidebar exposes a Top-K control allowing the user to adjust how many relevant chunks are retrieved and supplied to the LLM, within the supported range of 1–10 (default 5).

10.7 Backend Status

The frontend checks the `/health` endpoint and displays one of three states: **Connected**, **Checking**, or **Offline**, giving the user immediate feedback on whether the FastAPI backend is reachable.

10.8 Answer Display

The main area displays the project title, connection status, the ongoing conversation (user questions and RAG answers), and a question composer for submitting new questions.

10.9 Citations

Generated answers include citation markers that correspond to entries in the expandable Sources section, allowing the user to inspect exactly which retrieved chunks informed a given answer.

10.10 Performance Metrics

Each answer is displayed together with its associated performance metrics (retrieval latency, generation latency, and token counts), giving the user visibility into system performance alongside the answer content.

10.11 API Integration

The frontend communicates with the FastAPI backend over HTTP using two functions defined in `frontend/src/services/api.js`:

- `healthCheck()` → `GET /health`
- `askRag()` → `POST /query`

The default backend URL is `http://127.0.0.1:8000`. The frontend also measures client-side latency using `performance.now()`. The complete communication flow is:

```
React → api.js → HTTP POST → FastAPI → RAGPipeline → Response → React → Answer UI
```

CHAPTER 11 — TESTING AND RESULTS

11.1 Unit Testing

Individual testing scripts were written for each major pipeline component, in line with the project's modular design:

- Chunker (`test_chunker.py`)
- Embeddings (`test_embeddings.py`)
- Loader (`test_loader.py`)
- Retrieval (`test_retrieval.py`)
- Vector store (`test_vector_store.py`)

Testing each layer independently before integrating the full system made it easier to isolate and resolve issues early, rather than debugging failures across the entire pipeline at once.

11.2 Retrieval Testing

Retrieval was tested directly using `test_retrieval.py`, in addition to the formal evaluation described in Chapter 7.

11.3 Ground-Truth Evaluation

A ground-truth dataset of 20 questions was constructed and used to compute the retrieval metrics summarized in Chapter 7 (`build_ground_truth.py`, `evaluate_retrieval.py`).

11.4 API Testing

The FastAPI Swagger/OpenAPI interface was used to verify both the `/health` and `/query` endpoints, each returning HTTP 200 with the expected response structure (Chapter 9.8).

11.5 Frontend Testing

The React frontend was tested against the running FastAPI backend, confirming correct health-check display, question submission, answer rendering, and source inspection.

11.6 Integration Testing

Full frontend-backend integration was verified by submitting multiple questions through the UI and confirming that answers, citations, and performance metrics were correctly displayed end to end.

11.7 Out-of-Domain Testing

The system was tested with the out-of-domain question **"What is the capital of France?"**. Since the indexed documentation is Python documentation and contains no relevant geographical information, the system responded that the provided documentation does not

contain information about geography/capitals, and that the answer cannot be determined from the indexed sources.

This behavior is a desirable property of a grounded RAG system: it demonstrates that the system can recognize when the knowledge base does not support an answer, rather than blindly relying on unrelated retrieved context or fabricating a plausible-sounding but ungrounded response.

11.8 Final Results

Metric	Value
Documents processed	7
Chunks processed	363
Embedding dimension	384
Vector store count	363
Evaluation questions	20
Recall@1	0.5750
Recall@3	0.8667
Recall@5	0.9500
MRR	0.7375
nDCG@5	0.7896
`/health` endpoint	HTTP 200
`/query` endpoint	HTTP 200

The final frontend successfully communicated with FastAPI and correctly displayed generated answers, citations, and performance metrics for both in-domain and out-of-domain queries.

Example RAG results observed during testing:

#	Question	Retrieved Source	Result Summary
1	"What is a Python class?"	`classes.html`	Answer explained that a Python class is a user-defined type bundling data and functionality, referencing inheritance and methods.
2	"How does inheritance work in Python?"	`classes.html`	Answer explained base classes, derived classes, and multiple inheritance.
3	"What is a lambda expression?"	`controlflow.html`	Answer explained anonymous functions defined using the `lambda` keyword.
4	"How does Python handle exceptions?"	`errors.html`	Answer explained exception handling.
5	"How can a list be used as a stack?"	`datastructures.html`	Answer correctly explained `append()` and `pop()` using the LIFO (last-in, first-out) principle.

CHAPTER 12 — ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

12.1 Advantages

1. Cost efficiency — no paid external LLM API is required at query time.
2. Local inference via Ollama.
3. Source-grounded responses rather than answers based solely on model parametric knowledge.
4. Citation support, improving answer transparency.
5. Quantitative retrieval evaluation using standard IR metrics.
6. Modular architecture that is easy to test, debug, and extend.
7. Clean HTTP API exposed via FastAPI.
8. Professional web frontend built with React and Vite.
9. Adjustable Top-K retrieval.
10. Local data control — documents and queries remain on the local machine.
11. Independently testable components.
12. No dependency on paid external LLM APIs.

12.2 Limitations

The project is honest about its current constraints:

1. Knowledge is limited to the indexed documents; the system cannot answer questions outside this scope.

2. Retrieval quality depends on the chosen chunking strategy and embedding model.
3. Local LLM generation can be slower than a hosted, cloud-scale API.
4. Generation latency is affected by the host machine's hardware.
5. Citation formatting can still be improved.
6. Current chat history is frontend (React) state rather than persistent storage.
7. The system is currently designed and indexed specifically around the Python documentation corpus.
8. The system does not perform live web search.
9. The system does not automatically ingest new documents from the web.
10. Advanced reranking of retrieved chunks is not currently implemented.
11. Hybrid keyword + vector search is not currently implemented.
12. The system does not currently provide authentication or multi-user persistence.

12.3 Future Scope

The following items are identified as realistic future improvements and are explicitly **not** part of the current implementation:

- Hybrid search (keyword + vector).
- Reranking models applied to retrieved candidates.
- Improved citation mapping.
- Streaming LLM responses.
- Persistent conversation storage.
- User authentication.
- A document upload interface.
- PDF/DOCX ingestion support.
- Automatic document ingestion.
- Support for multiple knowledge bases.
- Larger and more extensive evaluation datasets.
- Larger local models.
- Model selection through the UI.
- GPU acceleration.
- Production deployment.
- Docker containerization.
- Monitoring and logging.
- Conversation memory across sessions.
- Improved source highlighting.

CHAPTER 13 — CONCLUSION

13.1 Final Summary

This project successfully demonstrates the complete development of a cost-efficient Retrieval-Augmented Generation system. The final solution integrates document processing, embeddings, a vector database, semantic retrieval, a local LLM, a FastAPI backend, and a React frontend into a single working pipeline:

```
Document Processing + Embeddings + Vector Database + Semantic Retrieval
+ Local LLM + FastAPI + React
```

The final system is complete and includes: document ingestion, chunking, embedding generation, ChromaDB vector storage, semantic retrieval, retrieval evaluation, the RAG pipeline itself, local LLM generation via Ollama, citation generation, performance measurement, a FastAPI backend, a React/Vite frontend, frontend-backend integration, and end-to-end testing. The system successfully answers questions from the indexed Python documentation using retrieved context and local LLM generation, and correctly declines to answer out-of-domain questions.

13.2 Project Outcome

The retrieval evaluation demonstrated a **Recall@5 of 95%**, indicating strong retrieval coverage within the top five retrieved chunks for the evaluated question set. The project also demonstrates that a genuinely useful RAG application can be built **without requiring a paid external LLM API**, by using Ollama for local inference — directly satisfying the project's core goal of cost efficiency while maintaining grounded, citation-backed answers.

REFERENCES

1. FastAPI Documentation — https://fastapi.tiangolo.com/
2. ChromaDB Documentation — https://docs.trychroma.com/
3. Ollama Documentation — https://ollama.com/
4. React Documentation — https://react.dev/
5. Vite Documentation — https://vitejs.dev/
6. Python Documentation (indexed knowledge base source) — https://docs.python.org/

APPENDIX

A. Deployment Commands (Local Development)

Backend:

```
uvicorn app.api:app --reload
```

Frontend:

```
npm run dev
```

Backend URL: `http://127.0.0.1:8000` Frontend: served via the Vite development server.

This represents a **local development deployment**, not a production deployment. Authentication, authorization, HTTPS, and other production-hardening measures would be required before deployment in a real multi-user environment (see Chapter 12.2, item 12).

B. Project Directory Structure

```
cost_efficient_rag/
│
├── app/
│   ├── __init__.py
│   ├── api.py
│   ├── chunker.py
│   ├── embeddings.py
│   ├── generator.py
│   ├── loaders.py
│   ├── rag.py
│   ├── retriever.py
│   ├── utils.py
│   └── vector_store.py
│
├── data/
├── documents/
│
├── evaluation/
│   └── ground_truth.json
│
├── frontend/
│   ├── node_modules/
│   ├── src/
│   │   ├── services/
│   │   │   └── api.js
│   │   ├── App.css
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── index.html
│   ├── package.json
│   ├── package-lock.json
│   └── vite.config.js
│
├── results/
│
└── scripts/
    ├── build_ground_truth.py
    ├── evaluate_retrieval.py
    ├── export_chunks.py
    ├── find_evaluation_chunks.py
    ├── get_chunk_id.py
    ├── ingest.py
    ├── test_chunker.py
    ├── test_embeddings.py
    ├── test_loader.py
```

```
    ├── test_retrieval.py
    └── test_vector_store.py
```

C. Development Challenges and Resolutions

During development, the following challenges were encountered and resolved:

- **API quota/credit limitations** when initially attempting to use the OpenAI API — resolved by switching to local inference via Ollama.
- **Import issues** involving the `Retriever` component — resolved through module restructuring.
- **Embedding dimension mismatch** with ChromaDB — resolved by aligning the embedding model's output dimension with the vector store configuration.
- **JSON formatting issues** during ground-truth evaluation — resolved by correcting the ground-truth file structure.
- **Frontend/backend connection issues** — resolved by verifying the backend URL configuration and endpoint availability.
- **npm/environment setup issues** — resolved during frontend environment setup.
- **FastAPI integration** — resolved through iterative testing via the Swagger/OpenAPI interface.
- **CORS/connectivity considerations**, addressed where applicable to allow the frontend to communicate with the backend.

These were development-stage challenges that were resolved during implementation; they do not represent failures in the final delivered system (see Chapter 13).

D. Security and Privacy Notes

Running inference locally through Ollama provides privacy advantages, since documents and queries remain on the local system and no paid external LLM API is required. However, the current implementation is a local development build and should not be considered production-secure. Authentication, authorization, HTTPS, and general production hardening would be necessary before deployment in a real multi-user environment.